

API Trading Platform - Partner Documentation

Table of Contents

1. [Overview](#)
2. [Response Structure](#)
3. [Base URL and Authentication](#)
4. [Security & IP Whitelisting](#)
5. [Orders](#)
 - [8.1 Place Order](#)
 - [8.2 Cancel Order](#)
 - [8.3 Modify Order](#)
6. [Portfolio](#)
 - [9.1 Get Holdings](#)
 - [9.2 Get Positions](#)
7. [Funds](#)
 - 10.1 [Get Funds](#)
8. [Margin Info](#)
 - 11.1 [Get Margin Info](#)
9. [Market Data](#)
 - 12.1 [Get LTP Data](#)
 - 12.2 [Get Market Depth Data](#)
 - 12.3 [Get Chart Data](#)
 - 12.4 [Get Historical Chart Data](#)

10. [Order Book & Trade Book](#)
 - 13.1 [Get User Order Book](#)
 - 13.2 [Get Order Log](#)
 - 13.3 [Get Trade Book](#)
11. [Transaction History](#)
 - 14.1 [Get Transaction History](#)
12. [Error Codes](#)
13. [Rate Limiting](#)
14. WebSocket Support (WIP)
15. [Best Practices](#)

Overview

The API Trading Platform provides a comprehensive REST API for trading Indian stocks, derivatives, and other financial instruments. The platform supports equity, futures, and options trading with real-time order management and portfolio tracking.

Key Features

- Multi-exchange trading (NSE, BSE, NFO, BFO)
 - Real-time order placement, modification, and cancellation
 - Portfolio and position tracking
 - Funds and margin management
 - Transaction history
 - WebSocket support for real-time updates
-

Response Structure

Success Response Format

All successful API responses follow this standard format:

```
{  
  "status": "success",  
  "message": "Human-readable success message",  
  "data": {  
    // Response data specific to the endpoint  
  }  
}
```

```
}
```

Success Response Fields:

Field	Type	Description
<code>status</code>	string	Always "success" for successful responses
<code>message</code>	string	Human-readable success message describing the operation
<code>data</code>	object	Contains the actual response data (structure varies by endpoint)

Error Response Format

All error responses follow this standard format:

```
{
  "status": "error",
  "message": "Human-readable error message",
  "error_code": "ERROR_CODE",
  "details": {
    // Additional error details (optional)
  }
}
```

Error Response Fields:

Field	Type	Description
<code>status</code>	string	Always "error" for error responses

<code>message</code>	string	Human-readable error message describing what went wrong
<code>error_code</code>	string	Machine-readable error code for programmatic handling
<code>details</code>	object	Additional error context (optional, may be null)

Common Error Codes

Error Code	Description	HTTP Status
<code>MISSING_REQUIRED</code>	Required field or header is missing	400 Bad Request
<code>VALIDATION_FAILED</code>	Input validation failed	400 Bad Request
<code>INVALID_API_KEY</code>	API key is invalid or expired	401 Unauthorized
<code>INVALID_ACCESS_TOKEN</code>	Access token is invalid or expired	401 Unauthorized
<code>AUTHENTICATION_FAILED</code>	Authentication failed (signature, IP, etc.)	401 Unauthorized
<code>IP_NOT_ALLOWED</code>	Request IP not in whitelist	401 Unauthorized
<code>INVALID_OTP</code>	OTP is incorrect or expired	401 Unauthorized

INVALID_PIN	PIN is incorrect	401 Unauthorized
INVALID_REQUEST_ID	Request ID not found or expired	401 Unauthorized
RATE_LIMIT_EXCEEDED	Too many requests in time period	429 Too Many Requests
INTERNAL_ERROR	Server error, try again later	500 Internal Server Error

HTTP Status Codes

Status Code	Description
200 OK	Request successful
400 Bad Request	Invalid request (missing fields, validation errors)
401 Unauthorized	Authentication failed
403 Forbidden	Access denied
404 Not Found	Resource not found
429 Too Many Requests	Rate limit exceeded

500 Internal Server Error	Server error
---------------------------	--------------

Base URL and Authentication

Base URL

`https://cs-prod.lemonn.co.in`

Required Headers

For all data APIs post access token generation:

Header	Required	Description
<code>Content-Type</code>	Yes	application/json
<code>x-api-key</code>	Yes	Your API key (public key)
<code>x-auth-key</code>	Yes	Access token for authentication

Security & IP Whitelisting

All API calls must originate from an IP address that is included in the whitelist provided during API key generation. Any request made from an IP address not present in the whitelist will be rejected with an authentication error. Ensure your server or application is running from a whitelisted IP.

- **IP Whitelisting is enforced for every API call.**
- **You can update your IP whitelist offline.**

Orders

1. Place Order

Endpoint: `POST /api-trading/api/v2/orders`

Description: Places a new order for buying or selling securities.

Headers:

- `Content-Type: application/json`
- `x-api-key: <api_key>`
- `x-auth-key: <access_token>`
- `x-client-id: <client_id>`

Request Body:

BUY:

```
{
  "clientId": "CLIENT123",
  "tag": "TAG001",
  "transactionType": "BUY",
  "exchangeSegment": "BSE",
  "productType": "DELIVERY",
  "orderType": "STOPLOSS",
  "validity": "DAY",
  "symbol": "RELIANCE",
  "securityId": "RELIANCE-A",
  "quantity": "100",
  "disclosedQuantity": "50",
  "price": "2500.50",
  "triggerPrice": "2450.00",
  "afterMarketOrder": false
}
```

```
{
  "clientId": "CLIENT123",
  "tag": "TAG003",
  "transactionType": "BUY",
  "exchangeSegment": "BFO",
  "productType": "DELIVERY",
  "orderType": "STOPLOSS",
}
```

```
"validity": "DAY",
"symbol": "SENSEX",
"contractType": "OPT",
"expiry": "01-01-2026",
"strikePrice": "84600",
"optionType": "CE",
"quantity": "20",
"disclosedQuantity": "20",
"price": "270.50",
"triggerPrice": "260.00",
"afterMarketOrder": false
}
```

SELL:

```
{
  "clientId": "CLIENT123",
  "tag": "TAG003",
  "transactionType": "SELL",
  "exchangeSegment": "BSE",
  "productType": "DELIVERY",
  "orderType": "STOPLOSS",
  "validity": "DAY",
  "symbol": "RELIANCE",
  "securityId": "RELIANCE-A",
  "quantity": "100",
  "disclosedQuantity": "50",
  "price": "2450.50",
  "triggerPrice": "2500.00",
  "afterMarketOrder": false
}
```

```
{
  "clientId": "CLIENT123",
  "tag": "TAG004",
  "transactionType": "SELL",
  "exchangeSegment": "BFO",
  "productType": "DELIVERY",
  "orderType": "STOPLOSS",
  "validity": "DAY",
  "symbol": "SENSEX",
  "contractType": "OPT",
  "expiry": "01-01-2026",
  "strikePrice": "84600",
  "optionType": "CE",
  "quantity": "20",
  "disclosedQuantity": "20",
  "price": "270.50",
  "triggerPrice": "260.00",
  "afterMarketOrder": false
}
```


Field Descriptions:

Field	Type	Required	Description
<code>clientId</code>	string	Yes	Client identifier
<code>tag</code>	string	No	Order tag for identification
<code>transactionType</code>	string	Yes	BUY or SELL
<code>exchangeSegment</code>	string	Yes	NSE, BSE, NFO, BFO
<code>productType</code>	string	Yes	DELIVERY, INTRADAY, MARGIN, MTF
<code>orderType</code>	string	Yes	MARKET, LIMIT, STOP_LOSS, STOP_LOSS_MARKET
<code>validity</code>	string	Yes	DAY, IOC, GTD
<code>symbol</code>	string	Yes	Trading symbol
<code>securityId</code>	string	No	Security identifier
<code>contractType</code>	string	Conditional	FUT or OPT (required for NFO/BFO)

<code>expiry</code>	string	Conditional	Expiry date in dd-mm-yyyy format (required for derivatives)
<code>strikePrice</code>	string	Conditional	Strike price (required for options)
<code>optionType</code>	string	Conditional	CE or PE (required for options)
<code>quantity</code>	string	Yes	Order quantity
<code>disclosedQuantity</code>	string	No	Disclosed quantity
<code>price</code>	string	Conditional	Order price (required for LIMIT orders)
<code>triggerPrice</code>	string	Conditional	Trigger price (required for STOP orders)
<code>afterMarketOrder</code>	boolean	No	Whether it's an AMO order

Validation Rules:

- For NFO/BFO segments: `contractType` and `expiry` are required
- For options (`contractType` = "OPT"): `strikePrice` and `optionType` are required
- `optionType` must be either "CE" or "PE"
- `contractType` must be either "FUT" or "OPT"

Success Response (200):

```
{
  "status": "success",
```

```
"message": "Order placed successfully",
"data": {
  "orderId": "ORD123456789",
  "orderStatus": "PENDING"
}
}
```

Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data.orderId</code>	string	Unique order identifier
<code>data.orderStatus</code>	string	Current order status (PENDING, FILLED, CANCELLED, etc.)

Error Responses:

Status Code	Error Code	Message	Description
400 Bad Request	MISSING_REQUIRED	<code>clientId is required</code>	Missing client ID field
400 Bad Request	MISSING_REQUIRED	<code>transactionType is required</code>	Missing transaction type

400 Bad Request	MISSING_REQUIRED	exchangeSegment is required	Missing exchange segment
400 Bad Request	MISSING_REQUIRED	productType is required	Missing product type
400 Bad Request	MISSING_REQUIRED	orderType is required	Missing order type
400 Bad Request	MISSING_REQUIRED	validity is required	Missing validity
400 Bad Request	MISSING_REQUIRED	symbol is required	Missing symbol
400 Bad Request	MISSING_REQUIRED	quantity is required	Missing quantity
400 Bad Request	VALIDATION_FAILED	contractType is required for NFO	Missing contract type for derivatives
400 Bad Request	VALIDATION_FAILED	invalid contractType: FUT. Must be FUT or OPT	Invalid contract type
400 Bad Request	VALIDATION_FAILED	expiry is required for NFO	Missing expiry for derivatives
400 Bad Request	VALIDATION_FAILED	strikePrice is required for options	Missing strike price for options

400 Bad Request	VALIDATION_FAILED	optionType is required for options	Missing option type
400 Bad Request	VALIDATION_FAILED	invalid optionType: CE. Must be CE or PE	Invalid option type
401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
403 Forbidden	AUTHORIZATION_FAILED	Market closed	Trading not allowed, market closed
500 Internal Server Error	INTERNAL_ERROR	Failed to place order	Service unavailable

2. Cancel Order

Endpoint: `POST /api-trading/api/v2/orders/cancel`

Description: Cancels an existing order.

Headers:

- `Content-Type: application/json`
- `x-api-key: <api_key>`
- `x-auth-key: <access_token>`
- `x-client-id: <client_id>`

Request Body:

```
{
  "orderId": "ORD123456789"
```

}

Field Descriptions:

Field	Type	Required	Description
<code>orderId</code>	string	Yes	Order ID to cancel

Success Response (200):

```
{
  "status": "success",
  "message": "Order cancelled successfully",
  "data": {
    "orderId": "ORD123456789",
    "orderStatus": "CANCELLED"
  }
}
```

Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data.orderId</code>	string	Unique order identifier
<code>data.orderStatus</code>	string	Updated order status (CANCELLED)

Error Responses:

Status Code	Error Code	Message	Description
400 Bad Request	MISSING_REQUIRE D	orderId is required	Missing order ID field
400 Bad Request	VALIDATION_FAIL ED	Order already cancelled	Order has already been cancelled
400 Bad Request	VALIDATION_FAIL ED	Order not found	Order ID does not exist
401 Unauthorized	AUTHENTICATION_ FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
403 Forbidden	AUTHORIZATION_F AILED	Order not cancellable	Order cannot be cancelled
500 Internal Server Error	INTERNAL_ERROR	Failed to cancel order	Service unavailable

3. Modify Order

Endpoint: `POST /api-trading/api/v2/orders/modify`

Description: Modifies an existing order parameters.

Headers:

- `Content-Type: application/json`
- `x-api-key: <api_key>`

- x-auth-key: <access_token>
- x-client-id: <client_id>

Request Body:

```
{
  "orderId": "ORD123456789",
  "quantity": "150",
  "price": "2550.00",
  "triggerPrice": "2500.00",
  "validity": "DAY"
}
```

Field Descriptions:

Field	Type	Required	Description
<code>orderId</code>	string	Yes	Order ID to modify
<code>quantity</code>	string	No	New quantity
<code>price</code>	string	No	New price
<code>triggerPrice</code>	string	No	New trigger price
<code>validity</code>	string	No	New validity

Success Response (200):

```
{
  "status": "success",
  "message": "Order modified successfully",
  "data": {
    "orderId": "ORD123456789",
    "orderStatus": "MODIFIED"
  }
}
```


Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data.orderId</code>	string	Unique order identifier
<code>data.orderStatus</code>	string	Updated order status (MODIFIED)

Error Responses:

Status Code	Error Code	Message	Description
400 Bad Request	MISSING_REQUIRED	<code>orderId is required</code>	Missing order ID field
400 Bad Request	VALIDATION_FAILED	Order not found	Order ID does not exist
400 Bad Request	VALIDATION_FAILED	Order not modifiable	Order cannot be modified
400 Bad Request	VALIDATION_FAILED	Invalid parameters	Invalid modification parameters

401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
403 Forbidden	AUTHORIZATION_FAILED	Order cannot be modified	Order modification not allowed
500 Internal Server Error	INTERNAL_ERROR	Failed to modify order	Service unavailable

Portfolio

1. Get Holdings

Endpoint: `GET /api-trading/api/v2/holdings`

Description: Retrieves current portfolio holdings.

Headers:

- `x-api-key: <api_key>`
- `x-auth-key: <access_token>`
- `x-client-id: <client_id>`

Query Parameters: None

Success Response (200):

```
{
  "status": "success",
  "message": "Holdings retrieved successfully",
  "data": {
    "holdings": [
      {
```

```

    "symbol": {
      "token": "500325",
      "symbol": "RELIANCE",
      "isin": "INE002A01018",
      "exchange": "BSE"
    },
    "quantity": 100,
    "averagePrice": 2500.50,
    "lastPrice": 2550.00,
    "investedValue": 250050.00,
    "currentValue": 255000.00,
    "profitAndLoss": 4950.00
  }
}
]
}
}

```

Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data.holdings</code>	array	Array of holding objects
<code>data.holdings[].symbol.token</code>	string	Symbol token identifier
<code>data.holdings[].symbol.symbol</code>	string	Trading symbol name
<code>data.holdings[].symbol.isin</code>	string	ISIN code of the security
<code>data.holdings[].symbol.exchange</code>	string	Exchange (NSE, BSE, etc.)

<code>data.holdings[].quantity</code>	integer	Number of shares held
<code>data.holdings[].averagePrice</code>	float	Average purchase price per share
<code>data.holdings[].lastPrice</code>	float	Last traded price
<code>data.holdings[].investedValue</code>	float	Total amount invested
<code>data.holdings[].currentValue</code>	float	Current market value
<code>data.holdings[].profitAndLoss</code>	float	Unrealized profit/loss

Error Responses:

Status Code	Error Code	Message	Description
401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
500 Internal Server Error	INTERNAL_ERROR	Failed to retrieve holdings	Service unavailable

2. Get Positions

Endpoint: `GET /api-trading/api/v2/positions`

Description: Retrieves current trading positions (intraday and carry forward).

Headers:

- x-api-key: <api_key>
- x-auth-key: <access_token>
- x-client-id: <client_id>

Query Parameters: None

Success Response (200):

```
{
  "status": "success",
  "message": "Positions retrieved successfully",
  "data": {
    "positions": [
      {
        "symbol": {
          "token": "500325",
          "symbol": "RELIANCE",
          "isin": "INE002A01018",
          "exchange": "BSE"
        },
        "quantity": 50,
        "buyQuantity": 100,
        "sellQuantity": 50,
        "buyValue": 250000.00,
        "sellValue": 125000.00,
        "averageBuyPrice": 2500.00,
        "averageSellPrice": 2500.00,
        "lastTradedTime": "2024-01-15T10:30:00Z",
        "isCarryForward": false,
        "isSquaredOff": false,
        "productType": "MIS",
        "exchangeTag": "BSE",
        "buyQuantityDaily": 100,
        "sellQuantityDaily": 50,
        "quantityDaily": 50,
        "buyValueDaily": 250000.00,
        "sellValueDaily": 125000.00,
        "averageBuyPriceDaily": 2500.00,
        "averageSellPriceDaily": 2500.00
      }
    ]
  }
}
```

Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data.positions</code>	array	Array of position objects
<code>data.positions[].symbol.token</code>	string	Symbol token identifier
<code>data.positions[].symbol.symbol</code>	string	Trading symbol name
<code>data.positions[].symbol.isin</code>	string	ISIN code of the security
<code>data.positions[].symbol.exchange</code>	string	Exchange (NSE, BSE, etc.)
<code>data.positions[].quantity</code>	integer	Net position quantity
<code>data.positions[].buyQuantity</code>	integer	Total buy quantity
<code>data.positions[].sellQuantity</code>	integer	Total sell quantity
<code>data.positions[].buyValue</code>	float	Total buy value

<code>data.positions[].sellValue</code>	float	Total sell value
<code>data.positions[].averageBuyPrice</code>	float	Average buy price
<code>data.positions[].averageSellPrice</code>	float	Average sell price
<code>data.positions[].lastTradedTime</code>	string	Last trade timestamp (ISO 8601)
<code>data.positions[].isCarryForward</code>	boolean	Whether position is carried forward
<code>data.positions[].isSquaredOff</code>	boolean	Whether position is squared off
<code>data.positions[].productType</code>	string	Product type (MIS, CNC, NRML)
<code>data.positions[].exchangeTag</code>	string	Exchange tag
<code>data.positions[].buyQuantityDaily</code>	integer	Daily buy quantity
<code>data.positions[].sellQuantityDaily</code>	integer	Daily sell quantity
<code>data.positions[].quantityDaily</code>	integer	Daily net quantity
<code>data.positions[].buyValueDaily</code>	float	Daily buy value

<code>data.positions[].sellValueDaily</code>	float	Daily sell value
<code>data.positions[].averageBuyPriceDaily</code>	float	Daily average buy price
<code>data.positions[].averageSellPriceDaily</code>	float	Daily average sell price

Error Responses:

Status Code	Error Code	Message	Description
401 Unauthorized	<code>AUTHENTICATION_FAILED</code>	Invalid access token	Access token not found or expired
401 Unauthorized	<code>INVALID_API_KEY</code>	Invalid API key	API key not found or expired
500 Internal Server Error	<code>INTERNAL_ERROR</code>	Failed to retrieve positions	Service unavailable

Funds

1. Get Funds

Endpoint: `GET /api-trading/api/v2/funds`

Description: Retrieves available funds and margin information.

Headers:

- x-api-key: <api_key>
- x-auth-key: <access_token>
- x-client-id: <client_id>

Query Parameters: None

Success Response (200):

```
{
  "status": "success",
  "message": "Funds retrieved successfully",
  "data": {
    "funds": {
      "creditForSale": "50000.00",
      "totalTradingPowerLimit": "100000.00",
      "product": "CNC",
      "limitUtilization": "50.00",
      "optionCfs": "25000.00",
      "mtmPAndL": "5000.00",
      "exposureMargin": "10000.00",
      "fundsTransferredToday": "5000.00",
      "notionalDeposit": "0.00",
      "netAvailableFunds": "45000.00",
      "bookedPAndL": "2000.00",
      "forAllocationOrWithdrawal": "40000.00",
      "cashDeposit": "50000.00",
      "deposit": "50000.00",
      "totalUtilization": "50000.00",
      "collateral": "0.00",
      "adhocDeposit": "0.00"
    }
  }
}
```

Response Field Descriptions:

Field	Type	Description
status	string	Response status ("success" or "error")
message	string	Human-readable success message

<code>data.funds.creditForSale</code>	string	Available credit for selling
<code>data.funds.totalTradingPowerLimit</code>	string	Total trading power limit
<code>data.funds.product</code>	string	Product type (CNC, MIS, NRML)
<code>data.funds.limitUtilization</code>	string	Percentage of limit utilized
<code>data.funds.optionCfs</code>	string	Options credit for sale
<code>data.funds.mtmPAndL</code>	string	Mark-to-market profit/loss
<code>data.funds.exposureMargin</code>	string	Exposure margin amount
<code>data.funds.fundsTransferredToday</code>	string	Funds transferred today
<code>data.funds.notionalDeposit</code>	string	Notional deposit amount
<code>data.funds.netAvailableFunds</code>	string	Net available funds

<code>data.funds.bookedPAndL</code>	string	Booked profit/loss
<code>data.funds.forAllocationOrWithdrawal</code>	string	Funds available for allocation/withdrawal
<code>data.funds.cashDeposit</code>	string	Cash deposit amount
<code>data.funds.deposit</code>	string	Total deposit amount
<code>data.funds.totalUtilization</code>	string	Total utilization amount
<code>data.funds.collateral</code>	string	Collateral amount
<code>data.funds.adhocDeposit</code>	string	Ad-hoc deposit amount

Error Responses:

Status Code	Error Code	Message	Description
401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired

401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
500 Internal Server Error	INTERNAL_ERROR	Failed to retrieve funds	Service unavailable

Margin Info

1. Get Margin Info

Endpoint: `POST /api-trading/api/v2/margin-info`

Description: Retrieves margin information for a specific order before placement. The system automatically converts date formats from dd-mm-yyyy to the internal database format for symbol lookup.

Headers:

- `Content-Type: application/json`
- `x-api-key: <api_key>`
- `x-auth-key: <access_token>`
- `x-client-id: <client_id>`

Request Body:

```
{
  "symbol": "NIFTY",
  "exchange": "NFO",
  "transactionType": "BUY",
  "price": "117.65",
  "quantity": "1",
  "productType": "MARGIN",
  "contractType": "OPT",
  "expiry": "31-07-2025",
  "strikePrice": "25100",
  "optionType": "PE"
}
```

Field Descriptions:

Field	Type	Required	Description
<code>symbol</code>	string	Yes	Trading symbol (e.g., NIFTY, RELIANCE)
<code>exchange</code>	string	Yes	Exchange segment (NSE, BSE, NFO, BFO)
<code>transactionType</code>	string	Yes	BUY or SELL
<code>price</code>	string	Yes	Order price
<code>quantity</code>	string	Yes	Order quantity
<code>productType</code>	string	Yes	Product type (MARGIN, DELIVERY)
<code>contractType</code>	string	Conditional	FUT or OPT (required for NFO/BFO)
<code>expiry</code>	string	Conditional	Expiry date in dd-mm-yyyy format (required for derivatives)

<code>strikePrice</code>	string	Conditional	Strike price (required for options)
<code>optionType</code>	string	Conditional	CE or PE (required for options)

Validation Rules:

- For NFO/BFO segments: `contractType` and `expiry` are required
- For options (`contractType = "OPT"`): `strikePrice` and `optionType` are required
- `optionType` must be either "CE" or "PE"
- `contractType` must be either "FUT" or "OPT"
- `transactionType` must be either "BUY" or "SELL"
- `productType` must be one of: "MARGIN", "DELIVERY"
- `expiry` must be in dd-mm-yyyy format (e.g., "31-07-2025")

Date Format Handling: The API automatically converts the expiry date from dd-mm-yyyy format to the internal database format for symbol lookup. For example:

- Input: "31-07-2025"
- Internal conversion: "31JUL25"
- Cache key: "NIFTY31JUL2525100PE_NFO"

Success Response (200):

```
{
  "status": "success",
  "message": "Margin info retrieved successfully",
  "data": {
    "approximateMargin": "208745.85",
    "marginMultiplier": "1",
    "availableMargin": "7458.86",
    "shortFall": "201286.99"
  }
}
```

Response Field Descriptions:

Field	Type	Description
-------	------	-------------

<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data.approximateMargin</code>	string	Approximate margin required for the order
<code>data.marginMultiplier</code>	string	Margin multiplier applied
<code>data.availableMargin</code>	string	Available margin in the account
<code>data.shortFall</code>	string	Margin shortfall (if any)

Error Responses:

Status Code	Error Code	Message	Description
400 Bad Request	MISSING_REQUIRED	<code>symbol is required</code>	Missing symbol field
400 Bad Request	MISSING_REQUIRED	<code>exchange is required</code>	Missing exchange field
400 Bad Request	MISSING_REQUIRED	<code>transactionType is required</code>	Missing transaction type
400 Bad Request	MISSING_REQUIRED	<code>price is required</code>	Missing price field

400 Bad Request	MISSING_REQUIRED	quantity is required	Missing quantity field
400 Bad Request	MISSING_REQUIRED	productType is required	Missing product type
400 Bad Request	VALIDATION_FAILED	contractType is required for NFO	Missing contract type for derivatives
400 Bad Request	VALIDATION_FAILED	invalid contractType: FUT. Must be FUT or OPT	Invalid contract type
400 Bad Request	VALIDATION_FAILED	expiry is required for NFO	Missing expiry for derivatives
400 Bad Request	VALIDATION_FAILED	strikePrice is required for options	Missing strike price for options
400 Bad Request	VALIDATION_FAILED	optionType is required for options	Missing option type
400 Bad Request	VALIDATION_FAILED	invalid optionType: CE. Must be CE or PE	Invalid option type
400 Bad Request	VALIDATION_FAILED	invalid transactionType: BUY. Must be BUY or SELL	Invalid transaction type
400 Bad Request	VALIDATION_FAILED	invalid productType: CNC. Must be MARGIN or DELIVERY	Invalid product type

400 Bad Request	VALIDATION_FAILED	invalid exchange: NSE. Must be NSE, BSE, NFO, or BFO	Invalid exchange
400 Bad Request	VALIDATION_FAILED	invalid date format: 31JUL25. Expected dd-mm-yyyy format	Invalid date format
400 Bad Request	VALIDATION_FAILED	symbol NIFTY not found with given criteria	Symbol not found in cache
401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
500 Internal Server Error	INTERNAL_ERROR	Failed to retrieve margin info	Service unavailable

Example Usage:

Equity Margin Info:

```
{
  "symbol": "RELIANCE",
  "exchange": "BSE",
  "transactionType": "BUY",
  "price": "2500.50",
  "quantity": "100",
  "productType": "MARGIN"
}
```

Futures Margin Info:

```
{
  "symbol": "NIFTY",
  "exchange": "NFO",

```

```
"transactionType": "SELL",
"price": "18500.00",
"quantity": "50",
"productType": "MARGIN",
"contractType": "FUT",
"expiry": "31-07-2025"
}
```

Options Margin Info:

```
{
  "symbol": "NIFTY",
  "exchange": "NFO",
  "transactionType": "BUY",
  "price": "117.65",
  "quantity": "75",
  "productType": "DELIVERY",
  "contractType": "OPT",
  "expiry": "31-07-2025",
  "strikePrice": "25100",
  "optionType": "PE"
}
```

Notes:

- The API automatically handles date format conversion for symbol lookup
- Ensure the strike price exists in your database for the given symbol and expiry
- The system uses internal cache keys in the format:

```
{SYMBOL}{EXPIRY}{STRIKEPRICE}{OPTIONTYPE}_{EXCHANGE}
```

Market Data

1. Get LTP Data

Endpoint: POST /api-trading/api/v2/market-data/ltp

Description: Retrieves Last Traded Price (LTP) and OHLC data for multiple symbols. Supports both security IDs (ntoken) and human-readable symbols.

Headers:

- Content-Type: application/json
- x-api-key: <api_key>
- x-auth-key: <access_token>
- x-client-id: <client_id>

Request Body:

```
{
  "NSE_EQ_TOKEN": [11536],
  "NSE_FNO_TOKEN": [49081, 49082],
  "NSE_EQ_SYMBOL": ["RELIANCE"],
  "NSE_FNO_SYMBOL": ["NIFTY_31-07-2025_25100_CE"]
}
```

Field Descriptions:

Field	Type	Required	Description
NSE_EQ_TOKEN	array	No	Array of NSE equity security IDs (ntoken)
BSE_EQ_TOKEN	array	No	Array of BSE equity security IDs (ntoken)
NSE_FNO_TOKEN	array	No	Array of NSE derivative security IDs (ntoken)
BSE_FNO_TOKEN	array	No	Array of BSE derivative security IDs (ntoken)
NSE_EQ_SYMBOL	array	No	Array of NSE equity symbols (e.g., ["RELIANCE"])

BSE_EQ_SYMBOL	array	No	Array of BSE equity symbols
NSE_FNO_SYMBOL	array	No	Array of NSE derivative symbols (e.g., ["NIFTY_31-07-2025_25100_CE"])
BSE_FNO_SYMBOL	array	No	Array of BSE derivative symbols

Validation Rules:

- At least one field must be provided
- Maximum 100 symbols can be requested at once
- Derivative symbols must be in format: SYMBOL_EXPIRY_STRIKE_OPTIONTYPE (e.g., "NIFTY_31-07-2025_25100_CE")
- Expiry date must be in dd-mm-yyyy format

Success Response (200):

```
{
  "status": "success",
  "message": "LTP data retrieved successfully",
  "data": {
    "11536": {
      "open": "1465.00",
      "high": "1476.00",
      "close": "1476.00",
      "low": "1423.10",
      "volume": "15213786",
      "last_price": "1425.90",
      "average_traded_price": "1438.82",
      "open_interest": "0",
      "change": "-50.10",
      "change_percentage": "-3.39"
    },
    "RELIANCE": {
      "open": "1465.00",
      "high": "1476.00",
      "close": "1476.00",
      "low": "1423.10",
      "volume": "15213786",
      "last_price": "1425.90",
      "average_traded_price": "1438.82",
      "open_interest": "0",
      "change": "-50.10",
      "change_percentage": "-3.39"
    }
  }
}
```

```

    "change_percentage": "-3.39"
  }
}

```

Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data</code>	object	Map of client input (token or symbol) to LTP data objects
<code>data[].open</code>	string	Day's open price
<code>data[].high</code>	string	Day's high
<code>data[].close</code>	string	Previous close price
<code>data[].low</code>	string	Day's low

<code>data[].volume</code>	string	Volume
<code>data[].last_price</code>	string	Last Traded Price
<code>data[].average_traded_price</code>	string	Average Traded Price
<code>data[].open_interest</code>	string	Open Interest
<code>data[].change</code>	string	Change from previous close
<code>data[].change_percentage</code>	string	Change percentage

Error Responses:

Status Code	Error Code	Message	Description
400 Bad Request	MISSING_REQUESTED	at least one symbol or security ID must be provided	No symbols provided in request
400 Bad Request	VALIDATION_FAILED	maximum 100 symbols can be requested at once	Too many symbols requested

400 Bad Request	VALIDATION_FAILED	invalid derivative symbol format	Invalid derivative symbol format
401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
500 Internal Server Error	INTERNAL_ERROR	Failed to get LTP data	Service unavailable

2. Get Market Depth Data

Endpoint: `POST /api-trading/api/v2/market-data/depth`

Description: Retrieves market depth (order book) data for multiple symbols. Supports both security IDs (ntoken) and human-readable symbols.

Headers:

- `Content-Type: application/json`
- `x-api-key: <api_key>`
- `x-auth-key: <access_token>`
- `x-client-id: <client_id>`

Request Body:

```
{
  "BSE_EQ_TOKEN": [500325],
  "BSE_FNO_TOKEN": [1144102],
  "BSE_EQ_SYMBOL": ["RELIANCE"],
  "BSE_FNO_SYMBOL": ["SENSEX_29-01-2026_86400_CE"]
}
```

Field Descriptions:

Field	Type	Required	Description
NSE_EQ_TOKEN	array	No	Array of NSE equity security IDs (ntoken)
BSE_EQ_TOKEN	array	No	Array of BSE equity security IDs (ntoken)
NSE_FNO_TOKEN	array	No	Array of NSE derivative security IDs (ntoken)
BSE_FNO_TOKEN	array	No	Array of BSE derivative security IDs (ntoken)
NSE_EQ_SYMBOL	array	No	Array of NSE equity symbols (e.g., ["RELIANCE"])
BSE_EQ_SYMBOL	array	No	Array of BSE equity symbols
NSE_FNO_SYMBOL	array	No	Array of NSE derivative symbols (e.g., ["NIFTY_31-07-2025_25100_CE"])
BSE_FNO_SYMBOL	array	No	Array of BSE derivative symbols

Validation Rules:

- At least one field must be provided
- Maximum 100 symbols can be requested at once

- Derivative symbols must be in format: `SYMBOL_EXPIRY_STRIKE_OPTIONTYPE` (e.g., "NIFTY_31-07-2025_25100_CE")
- Expiry date must be in dd-mm-yyyy format

Success Response (200):

```
{
  "status": "success",
  "message": "Market depth data retrieved successfully",
  "data": {
    "500325": {
      "sell": [
        {
          "orders": "3",
          "price": "1426.00",
          "quantity": "882"
        },
        {
          "orders": "8",
          "price": "1426.10",
          "quantity": "655"
        }
      ],
      "buy": [
        {
          "orders": "3",
          "price": "1425.90",
          "quantity": "8"
        },
        {
          "orders": "2",
          "price": "1425.80",
          "quantity": "48"
        }
      ],
      "total_buy_quantity": "885553",
      "total_sell_quantity": "1196794"
    },
    "RELIANCE": {
      "sell": [
        {
          "orders": "3",
          "price": "1426.00",
          "quantity": "882"
        }
      ],
      "buy": [
        {
          "orders": "3",
          "price": "1425.90",
          "quantity": "8"
        }
      ],
      "total_buy_quantity": "885553",
```

```

    "total_sell_quantity": "1196794"
  }
}

```

Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data</code>	object	Map of client input (token or symbol) to market depth data objects
<code>data[].sell</code>	array	Array of sell orders
<code>data[].sell[].orders</code>	string	Number of orders at this price level
<code>data[].sell[].price</code>	string	Sell price
<code>data[].sell[].quantity</code>	string	Total quantity at this price level

<code>data[].buy</code>	array	Array of buy orders
<code>data[].buy[].orders</code>	string	Number of orders at this price level
<code>data[].buy[].price</code>	string	Buy price
<code>data[].buy[].quantity</code>	string	Total quantity at this price level
<code>data[].total_buy_quantity</code>	string	Total buy quantity
<code>data[].total_sell_quantity</code>	string	Total sell quantity

Error Responses:

Status Code	Error Code	Message	Description
400 Bad Request	MISSING_REQUESTED	at least one symbol or security ID must be provided	No symbols provided in request
400 Bad Request	VALIDATION_FAILED	maximum 100 symbols can be requested at once	Too many symbols requested

400 Bad Request	VALIDATION_FAILED	invalid derivative symbol format	Invalid derivative symbol format
401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
500 Internal Server Error	INTERNAL_ERROR	Failed to get market depth data	Service unavailable

Example Usage:

Using Security IDs (ntoken):

```
{
  "BSE_EQ_TOKEN": [500325],
  "BSE_FNO_TOKEN": [1144102]
}
```

Using Human-Readable Symbols:

```
{
  "NSE_EQ_SYMBOL": ["RELIANCE"],
  "NSE_FNO_SYMBOL": ["SENSEX_29-01-2026_86400_CE"]
}
```

Mixed Request:

```
{
  "BSE_EQ_TOKEN": [500325],
  "BSE_EQ_SYMBOL": ["RELIANCE"],
  "BSE_FNO_SYMBOL": ["SENSEX_29-01-2026_86400_CE"]
}
```

Notes:

- The API automatically converts human-readable symbols to security IDs (ntoken) using the symbol cache
- For derivative symbols, the expiry date is automatically converted from dd-mm-yyyy to the internal format
- Redis keys are generated in the format: `LTP-{NTOKEN}-{EXCHANGE}` for LTP data and `OB-{NTOKEN}-{EXCHANGE}` for market depth data
- Data is retrieved from Redis in real-time
- Missing symbols are skipped and logged as warnings
- The response includes all available data for the requested symbols

3. Get Chart Data

Endpoint: `POST /api-trading/api/v2/market-data/chart`

Description: Retrieves intraday chart data for a single symbol with different time intervals. Supports both security IDs (ntoken) and human-readable symbols.

Headers:

- `Content-Type: application/json`
- `x-api-key: <api_key>`
- `x-auth-key: <access_token>`
- `x-client-id: <client_id>`

Request Body:

```
{
  "token": "500325",
  "exchange": "BSE",
  "interval": "1m",
  "start_time": "2025-06-25T13:13:00",
  "end_time": "2025-06-25T13:14:00"
}
```

Alternative Request Body (using symbol):

```
{
  "symbol": "RELIANCE",
  "exchange": "BSE",
  "interval": "1m",
  "start_time": "2025-06-25T13:13:00",
  "end_time": "2025-06-25T13:14:00"
}
```

Field Descriptions:

Field	Type	Required	Description
<code>token</code>	string	Conditional	Numeric token (e.g., "2885") - mutually exclusive with symbol
<code>symbol</code>	string	Conditional	Human-readable symbol (e.g., "RELIANCE" or "NIFTY_31-07-2025_25100_CE") - mutually exclusive with token
<code>exchange</code>	string	Yes	Exchange segment (NSE/1, BSE/3, NFO/2, BFO/4)
<code>interval</code>	string	Yes	Time interval (5s, 15s, 30s, 1m, 5m, 15m, 30m, 60m)
<code>start_time</code>	string	Yes	Start time in format "YYYY-MM-DDTHH:MM:SS"
<code>end_time</code>	string	Yes	End time in format "YYYY-MM-DDTHH:MM:SS"

	n g		
--	--------	--	--

Validation Rules:

- Either `token` or `symbol` must be provided (mutually exclusive)
- `token` must be numeric (e.g., "2885")
- `symbol` can be human-readable symbol (e.g., "RELIANCE") or derivative symbol
- `exchange` must be one of: NSE/1, BSE/3, NFO/2, BFO/4 (both string and numeric codes supported)
- `interval` must be one of: 5s, 15s, 30s, 1m, 5m, 15m, 30m, 60m
- `start_time` and `end_time` must be in ISO 8601 format
- For derivative symbols, use format: `SYMBOL_EXPIRY_STRIKE_OPTIONTYPE` (e.g., "NIFTY_31-07-2025_25100_CE")

Exchange Code Mapping:

- **1** = NSE (National Stock Exchange)
- **2** = NFO (NSE Futures & Options)
- **3** = BSE (Bombay Stock Exchange)
- **4** = BFO (BSE Futures & Options)

Success Response (200):

```
{
  "status": "success",
  "message": "Chart data retrieved successfully",
  "data": {
    "symbol": "500325",
    "exchange": "BSE",
    "interval": "1m",
    "start": "2025-06-25T13:13:00",
    "end": "2025-06-25T13:14:00",
    "points": [
      {
        "timestamp": "2025-06-25T13:13:00",
        "open": "1467.50",
        "high": "1468.70",
        "low": "1467.40",
        "close": "1468.10",
        "volume": "18841"
      },
      {
        "timestamp": "2025-06-25T13:14:00",
        "open": "1468.20",
        "high": "1468.30",
        "low": "1467.50",
        "close": "1467.80",
        "volume": "14880"
      }
    ]
  }
}
```

```
}
  ]
}
}
```

Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data.symbol</code>	string	Original symbol/token provided in request
<code>data.exchange</code>	string	Exchange segment
<code>data.interval</code>	string	Time interval used
<code>data.start</code>	string	Start time of the request
<code>data.end</code>	string	End time of the request
<code>data.points</code>	array	Array of chart candles
<code>data.points[].timestamp</code>	string	Candle timestamp

<code>data.points[].open</code>	string	Opening price
<code>data.points[].high</code>	string	Highest price
<code>data.points[].low</code>	string	Lowest price
<code>data.points[].close</code>	string	Closing price
<code>data.points[].volume</code>	string	Trading volume

Error Responses:

Status Code	Error Code	Message	Description
400 Bad Request	MISSING_REQUIRED	either token or symbol must be provided	Neither token nor symbol provided
400 Bad Request	MISSING_REQUIRED	both token and symbol cannot be provided. Use either token or symbol	Both token and symbol provided
400 Bad Request	MISSING_REQUIRED	exchange is required	Missing exchange field
400 Bad Request	MISSING_REQUIRED	interval is required	Missing interval field

400 Bad Request	MISSING_REQUIRED	start_time is required	Missing start time field
400 Bad Request	MISSING_REQUIRED	end_time is required	Missing end time field
400 Bad Request	VALIDATION_FAILED	token must be numeric: ABC	Invalid token format
400 Bad Request	VALIDATION_FAILED	invalid exchange: BSE. Must be one of: NSE(1), BSE(3), NFO(2), BFO(4)	Invalid exchange
400 Bad Request	VALIDATION_FAILED	invalid interval: 2m. Must be one of: [5s 15s 30s 1m 5m 15m 30m 60m]	Invalid interval
400 Bad Request	VALIDATION_FAILED	invalid time format: invalid start_time format	Invalid time format
400 Bad Request	VALIDATION_FAILED	symbol RELIANCE not found in cache for exchange NSE	Symbol not found
401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired

500 Internal Server Error	INTERNAL_ERR OR	Failed to get chart data	Service unavailable
------------------------------	--------------------	--------------------------	------------------------

Example Usage:

Using Token:

```
{
  "token": "500325",
  "exchange": "BSE",
  "interval": "1m",
  "start_time": "2025-06-25T13:13:00",
  "end_time": "2025-06-25T13:14:00"
}
```

Using Human-Readable Symbol:

```
{
  "symbol": "RELIANCE",
  "exchange": "BSE",
  "interval": "5m",
  "start_time": "2025-06-25T13:00:00",
  "end_time": "2025-06-25T13:30:00"
}
```

Using Derivative Symbol:

```
{
  "symbol": "NIFTY_31-07-2025_25100_CE",
  "exchange": "NFO",
  "interval": "15m",
  "start_time": "2025-06-25T09:00:00",
  "end_time": "2025-06-25T15:30:00"
}
```

Using Numeric Exchange Codes:

```
{
  "token": "2885",
  "exchange": "1",
  "interval": "1m",
  "start_time": "2025-06-25T13:13:00",
  "end_time": "2025-06-25T13:14:00"
}
```

```
}
```

```
{  
  "symbol": "NIFTY_31-07-2025_25100_PE",  
  "exchange": "2",  
  "interval": "5m",  
  "start_time": "2025-06-25T09:00:00",  
  "end_time": "2025-06-25T15:30:00"  
}
```

Notes:

- The API automatically converts human-readable symbols to security IDs (ntoken) using the symbol cache
- For derivative symbols, the expiry date is automatically converted from dd-mm-yyyy to the internal format
- The system internally calls the external chart API with the format: `{TOKEN}_{SEGMENT}` where segment mapping is: 1 for NSE, 2 for NFO, 3 for BSE, and 4 for BFO
- Both string (NSE, BSE, NFO, BFO) and numeric (1, 2, 3, 4) exchange codes are supported
- Chart data is retrieved from the external chart service in real-time
- The external API response is transformed from array format (c, h, l, o, v, t) to individual candle objects
- Timestamps are converted from milliseconds to ISO 8601 format
- Time intervals are validated against the allowed values
- The response includes the original request parameters for reference

4. Get Historical Chart Data

Endpoint: `POST /api-trading/api/v2/market-data/historical-chart`

Description: Retrieves historical chart data for a symbol. Response includes both `epochTime` and IST `timestamp` per candle.

Headers, request body, and validations mirror "Get Chart Data". The response contains an array of historical candles with `epochTime`, `timestamp`, `open`, `high`, `low`, `close`, `volume`.

Order Book & Trade Book

1. Get User Order Book

Endpoint: GET /api-trading/api/v2/orderbook

Description: Retrieves the authenticated user's order book (all orders for the day).

Headers:

- x-api-key: <api_key>
- x-auth-key: <access_token>
- x-client-id: <client_id>

Success Response (200):

```
{
  "status": "success",
  "message": "Orderbook retrieved successfully",
  "data": {
    "orders": [
      {
        "orderType": "RL",
        "reason": "",
        "isOrderOpen": true,
        "isModifiable": true,
        "orderTime": "2026-01-05 09:15:09",
        "exchOrdNo": "1300000000110194",
        "price": "1593.54",
        "errorReason": "",
        "isOrderAutoSquaredOff": false,
        "orderQty": 1,
        "modifiedBy": "",
        "exchOrderTime": "2025-08-11 09:15:09",
        "trigPrice": "0",
        "isAmoOrder": false,
        "productType": "DELIVERY",
        "initiatedBy": "",
        "validityDays": "0",
        "pendingQty": 0,
        "orderID": "AACKY00001;8",
        "symbol": {
          "token": "500325",
          "symbol": "RELIANCE",
          "isin": "INE002A01018",
          "exchange": "BSE"
        },
        "transactionType": "BUY",
        "underlyingStreamSym": "500325_BSE",
        "tradeQty": 1,
        "isCancellable": true,
        "validity": "DAY",
        "status": "EXECUTED",
        "discQty": 0
      }
    ]
  }
}
```

```

},
{
  "orderType": "RL",
  "reason": "",
  "isOrderOpen": true,
  "isModifiable": true,
  "orderTime": "2026-01-05 11:15:09",
  "exchOrdNo": "1300000003245235",
  "price": "1596",
  "errorReason": "",
  "isOrderAutoSquaredOff": false,
  "orderQty": 1,
  "modifiedBy": "",
  "exchOrderTime": "2025-08-11 09:15:09",
  "trigPrice": "0",
  "isAmoOrder": false,
  "productType": "DELIVERY",
  "initiatedBy": "",
  "validityDays": "0",
  "pendingQty": 0,
  "orderID": "AACKY00001;8",
  "symbol": {
    "token": "500325",
    "symbol": "RELIANCE",
    "isin": "INE002A01018",
    "exchange": "BSE"
  },
  "transactionType": "SELL",
  "underlyingStreamSym": "500325_BSE",
  "tradeQty": 1,
  "isCancellable": true,
  "validity": "DAY",
  "status": "EXECUTED",
  "discQty": 0
}, {
  "orderType": "RL",
  "reason": "",
  "isOrderOpen": true,
  "isModifiable": true,
  "orderTime": "2026-01-05 09:15:09",
  "exchOrdNo": "1300000000110195",
  "price": "600.54",
  "errorReason": "",
  "isOrderAutoSquaredOff": false,
  "orderQty": 20,
  "modifiedBy": "",
  "exchOrderTime": "2025-08-11 09:15:09",
  "trigPrice": "0",
  "isAmoOrder": false,
  "productType": "DELIVERY",
  "initiatedBy": "",
  "validityDays": "0",
  "pendingQty": 0,
  "orderID": "AACKY00001;8",
  "symbol": {

```

```

        "token": "1144102",
        "symbol": "SENSEX",
        "exchange": "BFO"
    },
    "transactionType": "BUY",
    "underlyingStreamSym": "1144102_BFO",
    "tradeQty": 20,
    "isCancellable": true,
    "validity": "DAY",
    "status": "EXECUTED",
    "discQty": 0
},
{
    "orderType": "RL",
    "reason": "",
    "isOrderOpen": true,
    "isModifiable": true,
    "orderTime": "2026-01-05 11:15:09",
    "exchOrdNo": "1300000003245235",
    "price": "679",
    "errorReason": "",
    "isOrderAutoSquaredOff": false,
    "orderQty": 20,
    "modifiedBy": "",
    "exchOrderTime": "2025-08-11 09:15:09",
    "trigPrice": "0",
    "isAmoOrder": false,
    "productType": "DELIVERY",
    "initiatedBy": "",
    "validityDays": "0",
    "pendingQty": 0,
    "orderID": "AACKY00001;8",
    "symbol": {
        "token": "1144102",
        "symbol": "SENSEX",
        "exchange": "BFO"
    },
    "transactionType": "SELL",
    "underlyingStreamSym": "1144102_BFO",
    "tradeQty": 20,
    "isCancellable": true,
    "validity": "DAY",
    "status": "EXECUTED",
    "discQty": 0
}
]
}
}

```

2. Get Order Log

Endpoint: `GET /api-trading/api/v2/order_log/{orderId}`

(see existing section above for details)

3. Get Trade Book

Endpoint: `GET /api-trading/api/v2/tradebook`

Description: Retrieves trades for the day. Optionally filter by `orderId`. Supports pagination via `limit` and `offset`.

Headers:

- `x-api-key: <api_key>`
- `x-auth-key: <access_token>`
- `x-client-id: <client_id>`

Query Parameters: | Parameter | Type | Required | Description | |-----|-----|-----|-----| |
`orderId` | string | No | Filter trades for a specific order ID (e.g., `AACKY00001;8`) | | `limit` | integer |
No | Items per page (default: 100) | | `offset` | integer | No | Page number starting at 1 (default: 1) |

If no trades exist for a requested page or filter, the API returns success with an empty `trades` array.

Success Response (200):

```
{
  "status": "success",
  "message": "Tradebook retrieved successfully",
  "data": {
    "trades": [
      {
        "transactionType": "BUY",
        "orderType": "RL",
        "underlyingStreamSym": "500325_BSE",
        "tradeNo": "600082373",
        "orderId": "AACKY00001;8",
        "exchOrdNo": "1300000000110194",
        "price": "1593.54",
        "tradeQty": 1,
        "symbol": {
          "token": "500325",
          "symbol": "RELIANCE",
          "isin": "INE002A01018",
          "exchange": "BSE"
        },
        "orderIdIdentifier": "AACKY2",
        "tradeTimeStamp": "2025-08-11 09:15:09",
        "productType": "DELIVERY"
      },
    ],
  },
}
```



```

    "transactionType": "BUY",
    "orderType": "RL",
    "underlyingStreamSym": "1144102_BFO",
    "tradeNo": "600082374",
    "orderID": "AACKY00001;8",
    "exchOrdNo": "1300000000110195",
    "price": "18.54",
    "tradeQty": 1,
    "symbol": {
      "token": "1144102",
      "symbol": "SENSEX",
      "exchange": "BFO"
    },
    "orderIdentifier": "AACKY 0",
    "tradeTimeStamp": "2025-08-11 09:15:09",
    "productType": "DELIVERY"
  },
  {
    "transactionType": "SELL",
    "orderType": "RL",
    "underlyingStreamSym": "500325_BSE",
    "tradeNo": "6000823901",
    "orderID": "AACKY00004;2",
    "exchOrdNo": "1300000000110194",
    "price": "1593.54",
    "tradeQty": 1,
    "symbol": {
      "token": "500325",
      "symbol": "RELIANCE",
      "isin": "INE002A01018",
      "exchange": "BSE"
    },
    "orderIdentifier": "AACKY3",
    "tradeTimeStamp": "2025-08-11 09:15:09",
    "productType": "DELIVERY"
  },
  {
    "transactionType": "SELL",
    "orderType": "RL",
    "underlyingStreamSym": "1144102_BFO",
    "tradeNo": "6000823214",
    "orderID": "AACKY00005;1",
    "exchOrdNo": "1300000000110195",
    "price": "18.54",
    "tradeQty": 1,
    "symbol": {
      "token": "1144102",
      "symbol": "SENSEX",
      "exchange": "BFO"
    },
    "orderIdentifier": "AACKY4",
    "tradeTimeStamp": "2025-08-11 09:15:09",
    "productType": "DELIVERY"
  }
}

```

```
}  
}
```

Transaction History

1. Get Transaction History

Endpoint: `GET /api-trading/api/v2/transaction_history`

Description: Retrieves transaction history for a date range.

Headers:

- `x-api-key: <api_key>`
- `x-auth-key: <access_token>`
- `x-client-id: <client_id>`

Query Parameters:

Parameter	Type	Required	Description
<code>from_date</code>	string	Yes	Start date in YYYY-MM-DD format
<code>to_date</code>	string	Yes	End date in YYYY-MM-DD format
<code>status</code>	string	Yes	Transaction status filter
<code>is_fno</code>	boolean	No	Filter for F&O transactions

Success Response (200):

```
{
  "status": "success",
  "message": "Transaction history retrieved successfully",
  "data": {
    "transactions": [
      {
        "transactionId": "TXN123456789",
        "utr": "UTR123456789",
        "accountNumber": "ACC123456",
        "amount": "50000.00",
        "time": "2024-01-15T10:30:00Z",
        "marketType": "EQUITY",
        "marketType2": "CASH",
        "paymentMethod": "BANK_TRANSFER",
        "transactionStatus": "COMPLETED",
        "transactionType": "DEPOSIT",
        "narration": "Fund deposit",
        "depositId": "DEP123456",
        "referencePayInId": "REF123456",
        "ctrCode": "CTR123"
      }
    ]
  }
}
```

Response Field Descriptions:

Field	Type	Description
<code>status</code>	string	Response status ("success" or "error")
<code>message</code>	string	Human-readable success message
<code>data.transactions</code>	array	Array of transaction objects

<code>data.transactions[].transactionId</code>	string	Unique transaction identifier
<code>data.transactions[].utr</code>	string	UTR (Unique Transaction Reference) number
<code>data.transactions[].accountNumber</code>	string	Account number
<code>data.transactions[].amount</code>	string	Transaction amount
<code>data.transactions[].time</code>	string	Transaction timestamp (ISO 8601)
<code>data.transactions[].marketType</code>	string	Market type (EQUITY, F&O, etc.)
<code>data.transactions[].marketType2</code>	string	Secondary market type (CASH, MARGIN, etc.)
<code>data.transactions[].paymentMethod</code>	string	Payment method (BANK_TRANSFER, UPI, etc.)
<code>data.transactions[].transactionStatus</code>	string	Transaction status (COMPLETED, PENDING, FAILED)
<code>data.transactions[].transactionType</code>	string	Transaction type (DEPOSIT, WITHDRAWAL, etc.)

<code>data.transactions[].narration</code>	string	Transaction description
<code>data.transactions[].depositId</code>	string	Deposit identifier
<code>data.transactions[].referencePayInId</code>	string	Reference pay-in identifier
<code>data.transactions[].ctrCode</code>	string	CTR (Cash Transaction Report) code

Error Responses:

Status Code	Error Code	Message	Description
400 Bad Request	MISSING_REQUIRED	<code>from_date is required</code>	Missing from date parameter
400 Bad Request	MISSING_REQUIRED	<code>to_date is required</code>	Missing to date parameter
400 Bad Request	MISSING_REQUIRED	<code>status is required</code>	Missing status parameter
400 Bad Request	VALIDATION_FAILED	Invalid date format	Date format should be YYYY-MM-DD

401 Unauthorized	AUTHENTICATION_FAILED	Invalid access token	Access token not found or expired
401 Unauthorized	INVALID_API_KEY	Invalid API key	API key not found or expired
500 Internal Server Error	INTERNAL_ERROR	Failed to retrieve transaction history	Service unavailable

Error Codes

Common Error Codes

Code	Description	HTTP Status
VALIDATION_FAILED	Request validation failed	400
AUTHENTICATION_FAILED	Invalid credentials	401
AUTHORIZATION_FAILED	Insufficient permissions	403
RESOURCE_NOT_FOUND	Requested resource not found	404
RATE_LIMIT_EXCEEDED	Too many requests	429

INTERNAL_SERVER_ERROR	Server error	500
-----------------------	--------------	-----

Trading-Specific Error Codes

Code	Description	HTTP Status
INSUFFICIENT_FUNDS	Insufficient funds for order	400
INVALID_SYMBOL	Invalid trading symbol	400
MARKET_CLOSED	Market is closed	403
ORDER_NOT_CANCELLABLE	Order cannot be cancelled	403
ORDER_NOT_MODIFIABLE	Order cannot be modified	403
INVALID_ORDER_TYPE	Invalid order type for symbol	400
QUANTITY_EXCEEDS_LIMIT	Order quantity exceeds limits	400

Authentication Error Codes

Code	Description	HTTP Status
INVALID_API_KEY	Invalid or expired API key	401

<code>IP_NOT_WHITELISTED</code>	IP address not in whitelist	401
<code>INVALID_SIGNATURE</code>	Invalid request signature	401
<code>EXPIRED_ACCESS_TOKEN</code>	Access token has expired	401
<code>INVALID_OTP</code>	Invalid or expired OTP	400
<code>INVALID_PIN</code>	Invalid trading PIN	400

Rate Limiting

The API implements rate limiting to ensure fair usage:

- **Registration APIs:** 5 requests per minute per IP
- **Authentication APIs:** 10 requests per minute per API key
- **Trading APIs:** 100 requests per minute per access token
- **Data APIs:** 200 requests per minute per access token

Rate limit headers are included in responses:

- `X-RateLimit-Limit`: Maximum requests per window
 - `X-RateLimit-Remaining`: Remaining requests in current window
 - `X-RateLimit-Reset`: Time when the rate limit resets
-

WebSocket Support

WIP (Work in Progress)

WebSocket functionality for real-time order updates and notifications is currently under development.

Best Practices

Security

- Store API keys and access tokens securely
 - Never expose private keys in client-side code
 - Use HTTPS for all API communications
 - Implement proper error handling for authentication failures
 - Monitor API usage for suspicious activity
 - Keep your IP whitelist updated
 - Implement proper session management
 - Secure your private key - it cannot be retrieved once lost
-